

AI Code Review & Security Analysis Report

Date: 2026-07-23 22:27:49 | Language: Python | Status: Rejected

Overall Review Status	Code Quality Score	Security Score	Total Findings
Rejected	76/100	30/100	13

Executive Summary & Severity Distribution

Critical: 3 | High: 1 | Medium: 2 | Low: 7

CRITICAL SECURITY RISK (30/100). Critical vulnerabilities detected that must be resolved immediately. Code quality is fair (76/100). Moderate refactoring is recommended for maintainability.

Detailed Findings

Issue	Severity	Line	Agent	Explanation
Variable 'x' is not descriptive	Low	5	Code Analysis	Single-letter variable name 'x' reduces code readability.
Variable 'a' is not descriptive	Low	7	Code Analysis	Single-letter variable name 'a' reduces code readability.
Variable 'b' is not descriptive	Low	8	Code Analysis	Single-letter variable name 'b' reduces code readability.
Variable 'c' is not descriptive	Low	9	Code Analysis	Single-letter variable name 'c' reduces code readability.
Function 'test_func' has too many parameters	Medium	6	Code Analysis	Function 'test_func' has 6 parameters, exceeding the threshold of 5.
Unused import 'os'	Low	2	Code Analysis	Import 'os' is defined on line 2 but never referenced.
Unused import 'pickle'	Low	3	Code Analysis	Import 'pickle' is defined on line 3 but never referenced.
Magic number '10' detected	Low	5	Code Analysis	Literal number 10 is hardcoded without explanation. Consider replacing it with a named constant.
Maintainability Score: 61.5/100	Medium	0	Code Analysis	Code maintainability score is 61.5/100 based on Halstead volume (85.8), total cyclomatic complexity (5), and lines of code (13).
Use of eval()	Critical	10	Security	The dynamic code execution function 'eval()' evaluates arbitrary strings as code, leading to arbitrary code execution.

Hardcoded credential: password	High	12	Security	Hardcoded secret or credential stored in variable 'password'. Move sensitive values to environment variables or key vaults.
SQL Injection in execute() call	Critical	13	Security	Dynamic query argument passed to execute(). Pass query parameters as a tuple/list in the second argument instead.
SQL Injection Vulnerability	Critical	13	Security	SQL query is constructed using string concatenation. Use parameterized queries (e.g., cursor.execute('SELECT * FROM t WHERE id = %s', (id,))) to prevent SQL injection.

Remediation Guidance & Refactoring

1. Variable 'x' is not descriptive (Line 5)

Problem: Single-letter or cryptic variable names obscure code intent, making code maintenance, debugging, and code reviews difficult.

Recommended Fix: Rename variables to descriptive, self-explanatory names matching Python PEP 8 conventions.

```
# Refactored descriptive variable names:
total_users = len(user_list)
for user_index in range(total_users):
    process_user(user_index)
```

Best Practice: Use meaningful, intention-revealing names for variables, functions, and classes following PEP 8 naming style guidelines.

References: PEP 8 - Style Guide for Python Code (Naming Conventions), Clean Code: Chapter 2 - Meaningful Names

2. Variable 'a' is not descriptive (Line 7)

Problem: Single-letter or cryptic variable names obscure code intent, making code maintenance, debugging, and code reviews difficult.

Recommended Fix: Rename variables to descriptive, self-explanatory names matching Python PEP 8 conventions.

```
# Refactored descriptive variable names:
total_users = len(user_list)
for user_index in range(total_users):
    process_user(user_index)
```

Best Practice: Use meaningful, intention-revealing names for variables, functions, and classes following PEP 8 naming style guidelines.

References: PEP 8 - Style Guide for Python Code (Naming Conventions), Clean Code: Chapter 2 - Meaningful Names

3. Variable 'b' is not descriptive (Line 8)

Problem: Single-letter or cryptic variable names obscure code intent, making code maintenance, debugging, and code reviews difficult.

Recommended Fix: Rename variables to descriptive, self-explanatory names matching Python PEP 8 conventions.

```
# Refactored descriptive variable names:
total_users = len(user_list)
for user_index in range(total_users):
    process_user(user_index)
```

Best Practice: Use meaningful, intention-revealing names for variables, functions, and classes following PEP 8 naming style guidelines.

References: PEP 8 - Style Guide for Python Code (Naming Conventions), Clean Code: Chapter 2 - Meaningful Names

4. Variable 'c' is not descriptive (Line 9)

Problem: Single-letter or cryptic variable names obscure code intent, making code maintenance, debugging, and code reviews difficult.

Recommended Fix: Rename variables to descriptive, self-explanatory names matching Python PEP 8 conventions.

```
# Refactored descriptive variable names:
total_users = len(user_list)
for user_index in range(total_users):
    process_user(user_index)
```

Best Practice: Use meaningful, intention-revealing names for variables, functions, and classes following PEP 8 naming style guidelines.

References: PEP 8 - Style Guide for Python Code (Naming Conventions), Clean Code: Chapter 2 - Meaningful Names

5. Function 'test_func' has too many parameters (Line 6)

Problem: Functions with excessive parameters (> 5) indicate high coupling, poor encapsulation, and make calls error-prone.

Recommended Fix: Group related parameters into a dataclass, Pydantic model, or parameter object.

```
# Parameter Object Refactoring:
from dataclasses import dataclass

@dataclass
class FilterOptions:
    category: str
    min_price: float
    max_price: float
    is_active: bool

def search_products(options: FilterOptions):
    # Logic using options object
    pass
```

Best Practice: Keep parameter lists short (<= 3-4 arguments). Pass cohesive data attributes wrapped in a typed dataclass.

References: Refactoring by Martin Fowler (Introduce Parameter Object), Clean Code: Function Arguments

6. Unused import 'os' (Line 2)

Problem: Unused import statements clutter module namespaces, increase startup overhead, and create unnecessary package dependencies.

Recommended Fix: Remove unused import statements from the top of the file.

```
# Clean module import header:
import os
# Removed unused module imports
```

Best Practice: Keep module imports clean and minimal. Use automated tools like autoflake, ruff, or black to automatically prune unused imports.

References: PEP 8 - Imports section, Ruff / Autoflake Linters

7. Unused import 'pickle' (Line 3)

Problem: Deserializing untrusted data using pickle, marshal, shelve, or yaml.load() allows arbitrary Python object reconstruction, leading directly to Remote Code Execution (RCE).

Recommended Fix: Replace unsafe deserializers with standardized, safe serialization formats like JSON (json.loads) or use yaml.safe_load().

```
# Secure JSON deserialization:
import json
data = json.loads(user_json_str)

# Secure YAML deserialization:
import yaml
config = yaml.safe_load(user_yaml_str)
```

Best Practice: Never deserialize untrusted data with pickle or marshal. Use safe, language-independent data exchange formats like JSON or Protocol Buffers.

References: OWASP Top 10: A08:2021 - Software and Data Integrity Failures, OWASP Deserialization Cheat Sheet, CWE-502: Deserialization of Untrusted Data, Python pickle Module Security Warning

8. Magic number '10' detected (Line 5)

Problem: Hardcoded numeric literals embedded in logic hide business meaning and make values difficult to update consistently.

Recommended Fix: Replace numeric literals with uppercase named constants at module or class scope.

```
# Named constants example:
MAX_RETRY_ATTEMPTS = 5
TIMEOUT_SECONDS = 30

if attempts > MAX_RETRY_ATTEMPTS:
    raise TimeoutError('Max retries exceeded')
```

Best Practice: Define descriptive, UPPERCASE named constants for numbers (other than standard 0, 1) to convey context and intent.

References: Refactoring by Martin Fowler (Replace Magic Number with Symbolic Constant), PEP 8 - Constants naming convention

9. Maintainability Score: 61.5/100 (Line 0)

Problem: High cyclomatic complexity means functions contain extensive branching and decision paths, leading to high bug risk and complex testing requirements.

Recommended Fix: Simplify decision trees by applying early guard clause returns or extracting decision branches into dedicated helper functions.

```
# Refactored with guard clauses:
def process_transaction(transaction):
    if not transaction.is_valid:
        return False
    if transaction.is_processed:
        return True
    return execute_transaction(transaction)
```

Best Practice: Maintain cyclomatic complexity ≤ 5 per function. Use guard clauses, lookup dictionaries, or polymorphic classes to replace complex nested conditionals.

References: McCabe Cyclomatic Complexity Metric, Refactoring: Replace Nested Conditional with Guard Clauses

10. Use of eval() (Line 10)

Problem: eval() and exec() parse and execute arbitrary text strings as Python code, providing an immediate path for arbitrary Remote Code Execution (RCE).

Recommended Fix: Remove eval()/exec() calls. Use safe data literal parsing with ast.literal_eval() or explicit dictionary mappings for operations.

```
# Secure literal parsing:
import ast
safe_data = ast.literal_eval(user_input_string)

# Secure dynamic dispatch:
ACTIONS = {'start': start_service, 'stop': stop_service}
if action in ACTIONS:
    ACTIONS[action]()
```

Best Practice: Avoid dynamic code execution. Parse structured data with standard serializers (JSON/YAML) or safe AST evaluation tools like ast.literal_eval().

References: OWASP Top 10: A03:2021 - Injection, CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection'), Python ast.literal_eval Documentation

11. Hardcoded credential: password (Line 12)

Problem: Hardcoding API keys, passwords, private keys, AWS secrets, or JWT secrets in code files exposes credentials in version control systems and built packages.

Recommended Fix: Extract secrets from source code files and load them dynamically from environment variables or a secret vault.

```
# Secure environment variable secret loading:
import os
api_secret = os.getenv('API_SECRET')
if not api_secret:
    raise RuntimeError('API_SECRET environment variable is missing')
```

Best Practice: Follow 12-Factor App principles. Keep secrets in environment variables or secret management services (AWS Secrets Manager, Vault) and include .env in .gitignore.

References: OWASP Top 10: A02:2021 - Cryptographic Failures, OWASP Secrets Management Cheat Sheet, CWE-798: Use of Hard-coded Credentials, The Twelve-Factor App: Config III

12. SQL Injection in execute() call (Line 13)

Problem: Constructing SQL queries by concatenating strings or formatting variables directly allows attackers to inject malicious SQL commands, leading to unauthorized database access, data theft, or database alteration.

Recommended Fix: Use parameterized queries (prepared statements) provided by your database library. Parameterized queries enforce strict separation between SQL code and user data.

```
# Secure parameterized query example:
cursor.execute('SELECT * FROM users WHERE username = %s AND status = %s', (username, status))

# Using SQLAlchemy ORM:
user = session.query(User).filter_by(username=username).first()
```

Best Practice: Never format or concatenate variables directly into SQL statements. Always use parameterized placeholders (%s, ?, :val) or an Object-Relational Mapper (ORM).

References: OWASP Top 10: A03:2021 - Injection, OWASP SQL Injection Prevention Cheat Sheet, CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection'), Python DB-API 2.0 Specification (PEP 249)

13. SQL Injection Vulnerability (Line 13)

Problem: Constructing SQL queries by concatenating strings or formatting variables directly allows attackers to inject malicious SQL commands, leading to unauthorized database access, data theft, or database alteration.

Recommended Fix: Use parameterized queries (prepared statements) provided by your database library. Parameterized queries enforce strict separation between SQL code and user data.

```
# Secure parameterized query example:  
cursor.execute('SELECT * FROM users WHERE username = %s AND status = %s', (username, status))
```

```
# Using SQLAlchemy ORM:  
user = session.query(User).filter_by(username=username).first()
```

Best Practice: Never format or concatenate variables directly into SQL statements. Always use parameterized placeholders (%s, ?, :val) or an Object-Relational Mapper (ORM).

References: OWASP Top 10: A03:2021 - Injection, OWASP SQL Injection Prevention Cheat Sheet, CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection'), Python DB-API 2.0 Specification (PEP 249)

Pull Request Summary & Next Steps

Top Priority Risks:

- Line 10 [Critical]: Use of eval() - The dynamic code execution function 'eval()' evaluates arbitrary strings as code, leading to arbitrary code execution.
- Line 13 [Critical]: SQL Injection in execute() call - Dynamic query argument passed to execute(). Pass query parameters as a tuple/list in the second argument instead.
- Line 13 [Critical]: SQL Injection Vulnerability - SQL query is constructed using string concatenation. Use parameterized queries (e.g., cursor.execute('SELECT * FROM t WHERE id = %s', (id,))) to prevent SQL injection.
- Line 12 [High]: Hardcoded credential: password - Hardcoded secret or credential stored in variable 'password'. Move sensitive values to environment variables or key vaults.
- Line 6 [Medium]: Function 'test_func' has too many parameters - Function 'test_func' has 6 parameters, exceeding the threshold of 5.

Recommended Next Steps:

1. Immediately resolve Critical severity security vulnerabilities (SQL Injection in execute() call, Use of eval()).
1. Remediate 1 High-severity issues prior to merging PR.
1. Address 9 Medium/Low maintainability and code quality suggestions.
1. Apply provided automated code remediation refactoring examples.